M T W T F S S
Page No.
Date:
YOUVA

5. Replication, Consistency & Fault tolerance

Q.1 Discuss design and implementation issues of DSM.

⇒

- Distributed Shared Memory (DSM) is a model where physically distributed memory is made to appear as a single logical shared memory to applications.
- DSM is a mechanism that manages memory across multiple nodes and makes inter-process communications transparent to end-users.
- Issues to Design and Implement DSM:
  - 1) Granularity
  - 2) structure of shared memory space
  - 3) Memory coherence and access synchronization
  - 4) Data location and access
  - 5) Replacement strategy
  - 6) Thrashing
  - 7) Heterogeneity

1) Granularity

⇒ Granularity refers to the block size of a DSM system.

- Granularity refers to the unit of sharing and the unit of data moving across the network when a network block shortcoming then we can utilize the estimation of block size as words/phrases.
- The block size might be different for the various networks.
- The amount of traffic generated due to network fault is also related with block size.

- Hence, it is necessary to select a proper block size.

2) Structure of shared memory space

⇒ Structure refers to the design of the shared data in memory.

- The structure of the shared memory space of a DSM system is regularly dependent on the sort of applications that the DSM system is intended to support.

3) Memory coherence and Access Synchronization

⇒ In the DSM system, the shared data things ought to be accessible by different nodes simultaneously in the network.

- The fundamental issues in this system is data Irregularity.
- The data Irregularity might be raised by the synchronous access.
- To solve this problem in the DSM system we need to utilize some synchronization primitives, semaphores, event count and so on.

4) Data location and access

⇒ It is necessary to locate and retrieve data accessed by a user process.

- This is required to share data.

- Hence, data block locating mechanism is needed to service network data block faults to fulfill the memory coherence semantics.

5) Replacement Stratergy

⇒ In the local memory of the node is full, a cache miss at the node implies not
- If cache in a node is full and there is a cache miss then existing data block in memory needs to be replaced to accomodate the newly fetched data block.
- Hence, it is important to have a cache replacement policy in the design of DSM system.

6) Thrashing

⇒ If simultaneously two nodes compete for write access to the same data item then the related data block might be moved back and forth at a such a high rate that no real work can get done.
- This situation is called thrashing and some policy is required to avoid it.

7) Heterogeneity

⇒ The DSM system worked for homogenous system and need ~~not~~ address the heterogeneity issues.
- In any case, assuming the underlined system is heterogenous, the DSM

system should be designed to deal with heterogenous, so it works appropriately with machines having different architectures.

Q.2 Write a short note on Replication and the types of it.

⇒

- Replication in distributed system refers to the process of creating and maintaining multiple copies of data, resources or services across different nodes within a network.
- The primary goal of replication is to enhance system reliability, availability and performance by ensuring that data or service are accessible even if some nodes fails or become unavailable.
- Replication plays important role in distributed systems due to several important reasons:

1) Enhanced availability

⇒ By replicating data or services across multiple nodes in a distributed system, you ensure that even if some nodes fails or becomes unreachable, the system as a whole remains available.

- Users can still access data or services from other healthy replicas.

2) Improved Reliability

⇒ Replication increases reliability by reducing the likelihood of a single point of failure.

- If one replica fails, others can continue to serve requests, maintaing system operations without interruption.

3) Reduced latency

⇒ Replicating data closer to users or clients can reduce latency, or delay the data transmission.

- This is particularly important in distributed systems serving users across different geographic locations.

4) Scalability

⇒ Replication supports scalability by distributing the workload across multiple nodes.

- As the demand of services or resources increases, additional replicas can be deployed to handle increased traffic or data processing requirements.

- Types
  - 1) Primary Backup Replication
  - 2) Multi-primary Replication
  - 3) Chain Replication
  - 4) Distributed Replication.

1) Primary Backup Replication

⇒ Primary-backup replication involves designating one primary replica to handle all updates, while one or more backup replicas maintain copies of the data and synchronize with the primary.

- Advantages:
a) Strong consistency: Since all updates go through the primary replica, read operations can be served with strong consistency guarantees.
b) Fault tolerance: If the primary replica fails, one of the backup replica can be promoted to become the new primary, ensuring continuous availability.
- Disadvantages:
a) Recource Utilization: Backup replicas are often idle unless a failover occurs, which can be seen as inefficient resource utilization.

2) Multi-primary Replication
⇒ Multi-primary replication allows multiple replicas to accept updates independently.
- Each replica acts as both a client and a server.
- Advantages:
a) Increased throughput: Multiple replicas can handle write requests concurrently, improving overall system throughput.
b) Fault tolerance.
- Disadvantages:
a) Conflict Resolution: Concurrent updates across multiple primaries can lead to conflicts that need to be resolved, typically using techniques like conflict detection and resolution algorithms.

3) Chain Replication

⇒ Chain replication involves replicating data sequentially through a chain of nodes.

- Each node in the chain forwards updates to the next node in the sequence, typically ending with a return path to the primary node.

- Advantage:

a) Fault tolerance

b) Strong Consistency.

- Disadvantage:

a) Performance: The overall performance of the system can be limited by the slowest node in the chain, as each update must traverse through every node in sequence.

4) Distributed Replication

⇒ Distributed replication distributes data or services across multiple nodes in a less structured manner compared to primary-backup or chain replication.

- Advantages:

a) Scalability

b) Fault tolerance

- Disadvantages:

a) Complexity

b) Consistency challenges

Q.3 Explain any five data-centric consistency models.

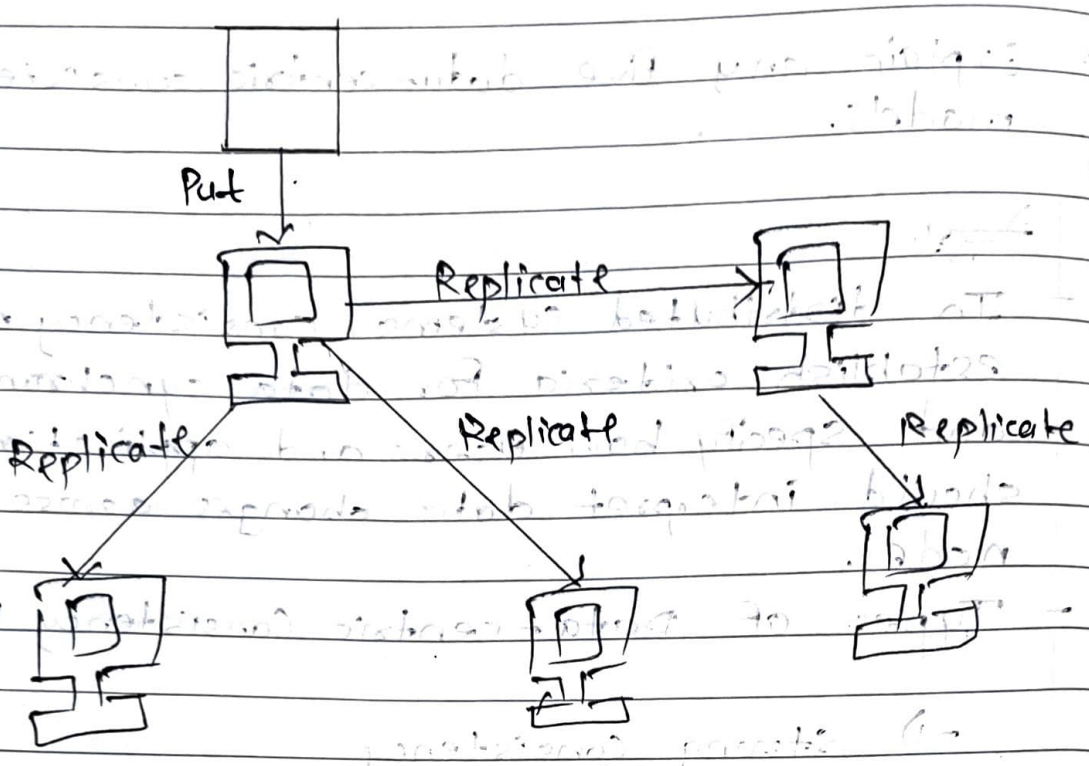
⇒

- In distributed systems, consistency models establish criteria for data synchronization and specify how users and applications should interpret data changes across several nodes.
- Types of Data-centric Consistency Models:
  - 1) Strong Consistency
  - 2) Sequential Consistency
  - 3) Causal Consistency
  - 4) FIFO Consistency
  - 5) Release Consistency

1) Strong Consistency

⇒ In a strongly consistent model, all nodes in the system agree on the order in which operations occured.

- Reads will always return the most recent version of data; when an update occurs on one server, this model makes sure every other server in the system replicates this change immediately.
- This model provides highest level of consistency but it can be slower and requires more resources in a distributed environment since all servers must stay perfectly in sync.



2) Sequential Consistency

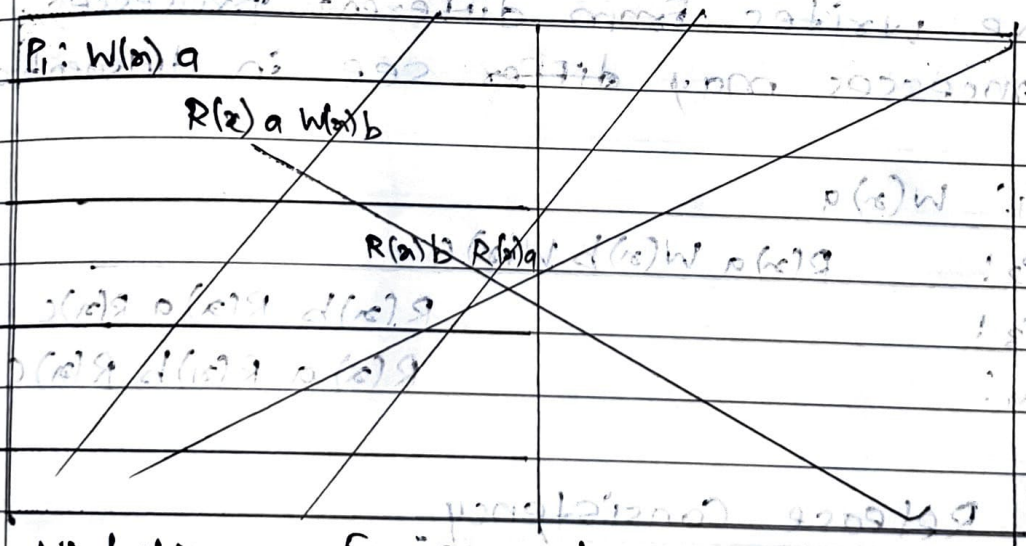
⇒ It is a consistency model in distributed system that ensures all operations across processes appear in a single, unified order.

- In this model, every read and write operations from any process appears to happen in sequence, regardless of where it occurs in the system.
- Importantly, all processes observe this same sequence of operations, maintaining a sense of consistency and order across the system.
- Example Data Stores:
  Shared memory emulations.
- E.g., Messaging Apps

3) Causal Consistency

⇒ The causal consistency model is a type of consistency in distributed systems that ensures that related events happen in a logical order.

- In simpler terms, if two operations are causally related, the system will make sure they are seen in that order by all users.
- However, if there's no clear relationship between two operations, the system doesn't enforce an order, meaning different users might see the operation in different sequences.



Violation of causal consistency

P1 : W(x) a
P2 : R(x) a W(x) b
P3 : R(x) a R(x) b ✓
P4 : R(x) b R(x) a ✗

- A casual consistent system:

P_1:	W(x) a		
P_2:		W(x) b	
P_3:	R(x) a	R(x) b	✓
P_4:	R(x) b	R(x) a	✓

4) FIFO consistency

⇒ FIFO consistency is the next step in relaxing the consistency by dropping requirement of seeing the casually related writes in same order at all processes.

- All the processes must see the writes done by single process in the order in which they were issued.
- The writes from different processes, processes may differ see in different orde

P_1:	W(x) a		
P_2:	R(x) a	W(x) b	W(x) c
P_3:		R(x) b	R(a) a R(x) c
P_4:		R(a) a	R(x) b R(x) c

5) Release Consistency

⇒ In release consistency model, acquire operation is used to tell the data store that CS is about to enter and release operation used to tell that, CS has just been exited.

- These operations are used to protect the shared data items.
- The shared data that kept consistent are said to be protected.

P1: Acq(L) W(x)a W(x)b Rel(L)

P2: Acq(L) R(x)b Rel(L)

P3: R(x)a

6) Weak Consistency

- A weakly consistent system provides no guarantee about the ordering of operations or the state of the data at any given time
- Clients may see different versions of the data depending on which node they connect to.
- This model provides the highest availability and scalability but at the cost of consistency.
- For eg., News channel

Q.4 Discuss and differentiate various client consistency models.

⇒

- Different client-consistency models:
  - 1) Monotonic Reads
  - 2) Monotonic Writes
  - 3) Read Your Writes
  - 4) Writes Follow Reads

1) Monotonic Reads

⇒ If a process reads the value of a data item x, any successive read operation on x by that process will always return that same value or a more recent value.

- This ensures that in later read operation, process will always get latest or same value and not older one.
- Consider the example of distributed email database.
- In this case, mailboxes of users are distributed and replicated on multiple machines.
- The received mail at any location is propagated in lazy manner to other copies of mailbox.
- Only that data gets forwarded which is needed to maintain consistency.
- Suppose users reads the mailbox in Mumbai and later flies to Delhi

- The Monotonic-read consistency guarantees that, the message in mailbox that was read at Mumbai will also be in mailbox at Delhi.

2) Monotonic Writes

⇒ In this model, it is important to propagate write operations in the correct order to all copies data store.
- Monotonic-read consistency is guaranteed by data store if following condition holds:
- A write operation by a process on a data item x is completed before any successive write operation on x by the same process.
- This ensures that, result of current write operation by the process on data item reflects the effect of previous effect operation by the same process on same data item.
- Consider the updating the software library. In this case, changes are always carried out in one or more functions.
- With monotonic writesconsistency promise is given that, whenever changes will be carried out at other copy of the library, all previous updates will be carried out first.

3) Read-Your-Writes

⇒ Read Your Writes model is closely related to monotonic reads model.

- Reads-your-writes consistency is guaranteed by data store if following condition holds.
- The result of a write operation by a process on data item x will always be seen by a successive read operation on x by the same process.
- This ensures that; write operation is always completed before a successive read operation by the same process, regardless of where that read operation occurs.
- Consider the example of changing password of digital library account on web.
- It takes some time to come in to effect this change. As a result, library may be inaccessible to the user.
- A reason for delay is use of separate server to manage passwords and it may take some time to subsequently propagate encrypted passwords to the various servers that form the library.

4) Write-Follows reads

⇒ In this consistency model updates of last read operations gets propagated.

- Write-Follows reads consistency is guaranteed by data store if following condition holds:
- A write operation by a process on a data item x following a previous read operation on x by the same process is guaranteed to take place on the same or a more recent value of x that was used.
- Consider the example of posting articles and their reactions in network newsgroup.
- User first reads the article X and then posts its reaction Y.
- As per write-follows-reads consistency, reaction Y will be written to anti copy ~~of~~ of the newsgroup only after article X has been written as well.

Q.5 What is fault tolerance? Explain failure models..

⇒

- Fault tolerance is defined as the ability of the system to function properly even in the presence of any failure.
- Distributed systems consist of multiple components due to which there is a high risk of faults occuring.
- Due to the presence of faults, the overall performance may degrade.
- Fault tolerance is required in order to provide below four features:

1) Availability

⇒ Availability is defined as the property where the system is readily available for its use at any time.

2) Reliability

⇒ Reliability is defined as the property where the system can work continuously without any failure.

3) Safety

⇒ Safety is defined as the property where the system can remain safe from unauthorized access even if any failure occurs.

4) Maintainability

⇒ Maintainability is defined as the property state that how easily and fastly the failed node or

system can be repaired.

- Types of Fault tolerance :

1) Hardware Fault Tolerance

⇒ Hardware Fault Tolerance involves keeping a backup plan for hardware devices such as memory, hard disk, CPU and other hardware peripheral devices.

2) Software Fault Tolerance

⇒ This is a type of fault tolerance where dedicated software is used in order to detect invalid output, runtime and programming errors.

3) System Fault Tolerance

⇒ System Fault tolerance is a type of Fault tolerance that consists of a whole system.

- Failure Models :

- In distributed system, things can go wrong, causing what we call failures.
- These failures are like hiccups in the system's functioning.
- Failure models help us in categorizing different ways things can go wrong.
- This classification is vital for system designers as it helps them prepare for potential issues.

- Models:
  - 1) Crash Failure
  - 2) Omission Failure
  - 3) Arbitary Failure
  - 4) Timing Failure
  - 5) Response Failure

1) Crash Failure

⇒ A crash failure occurs when a server abruptly stops working but was previously operational.

- A significant characteristics of crash failure is that nothing is heard from the server after it has stopped.
- Crash failure can lead to data loss or inconsistency if not handle properly.
- Systems employ techniques like redundancy and checkpointing to recover from crash failure.

2) Omission Failure

⇒ Omission Failure refers to the faults that occur due to failure of process or communication channel.

- Because of this failure, process or communication channel fails to carry out the action or task that it is supposed to do.
- A receiver omission failure may mean the server never received the request.

- A send omission failure occurs when the server completes its task but fails to respond.

3) Arbitary Failure

⇒ In this type of failure, process or communication channel behaves arbitarily.
- In this failure, the responding process may return wrong values or it may set wrong value in data item.
- Process may arbitarily omit intentional processing step or carry out unintentional processing step.

4) Timing Failure

⇒ In synchronous distributed system, limits are ~~sent to site~~ set on process execution time, message delivery time and clock drift rate.
- Hence, timing failure are applicable to this system.
- In timing failure, clock failure affects process as its local clock may drift from perfect time or may exceeds bound on its rate.

5) Response Failure

⇒ Response failure occurs when the server responds incorrectly, are serious.
- Two types of response failures can occur.
- In a value failure, a server sends the erroneous response.

- state transmission failure are ~~a server~~ other form of response failure.
- This failure occurs when the server responds unexpectedly to a request.

Q.6 How monotonic read consistency ~~and~~ model different then Redd Your Write Consistency model?

⇒

Monotonic read consistency	Redd Your Write Consistency
1) Once a process reads a version of data, it will never see an older version of that data in subsequent reads.	1) After a process writes a value to data, it will always see that same value in future reads.
2) Prevents the process from seeing data that appears to go back in time.	2) Ensures that process's own updates are visible in its future operations
3) This is a read-after-read guarantee.	3) This is a write-followed-by-read guarantee.
4) It maintains a logical progression of reads.	4) It maintains consistency between a write and the next read.
5) E.g., Reading timeline, Feed or analytical dashboard.	5) For eg; Editing a user profile, setting or draft.

Page No.:
Date:

Q.5 Explain independent checkpointing and coordinated checkpointing.

=>

- Checkpointing is a technique used in distributed systems for fault tolerance.
- It involves saving the state of process at certain points so that, in case of failure, the system can resume execution from the last checkpoint instead of restarting from the beginning.

1) Independent checkpointing

=> Each process in a distributed system takes checkpoints independently, without coordinating with other processes.

- Each process periodically saves its local state.
- If a failure occurs, a process restores its last checkpoint and continue execution.
- For e.g.,

Process P_1 saves its state at T_1, T_4, T_7
Process P_2 saves its state at T_2, T_5, T_8
If P_1 crashes at T_6, it restores T_4, but P_2 is already at T_5
If P_1 depends on data from P_2's state at T_5, inconsistency occurs.

- Advantages :-
  1) No synchronization overhead.
  2) Processes can take checkpoints at any time.

- Disadvantages :
i) Some processes may restore older states, leading to rollback propagation (domino effect)

2) Coordinated checkpointing
=> In coordinated checkpointing, all processes synchronize to write their state to local stable storage in cooperative manner.
- As a result of which, the saved state is automatically globally consistent.
- In this way, domino effect is avoided.
- A two-phase blocking protocol is used to coordinate checkpointing.
- A coordinator first multicast CHECKPOINT_REQUEST to all the processes.
- Receiving processes of this message then takes local checkpoint, queues any successive message handed to it by application it is running and acknowledges to the coordinator that it has taken a checkpoint.
- When a coordinator receives acknowledgement from all the processes, it multicast CHECPOINT_DONE message to permit blocked processes to carry on their work.
- Advantages :-
1) No rollback propagation
2) Simplifies recovery.

- Disadvantages :-
i) Coordination becomes complex in large distributed systems.

Q.1 Explain strict and weak consistency models with examples.

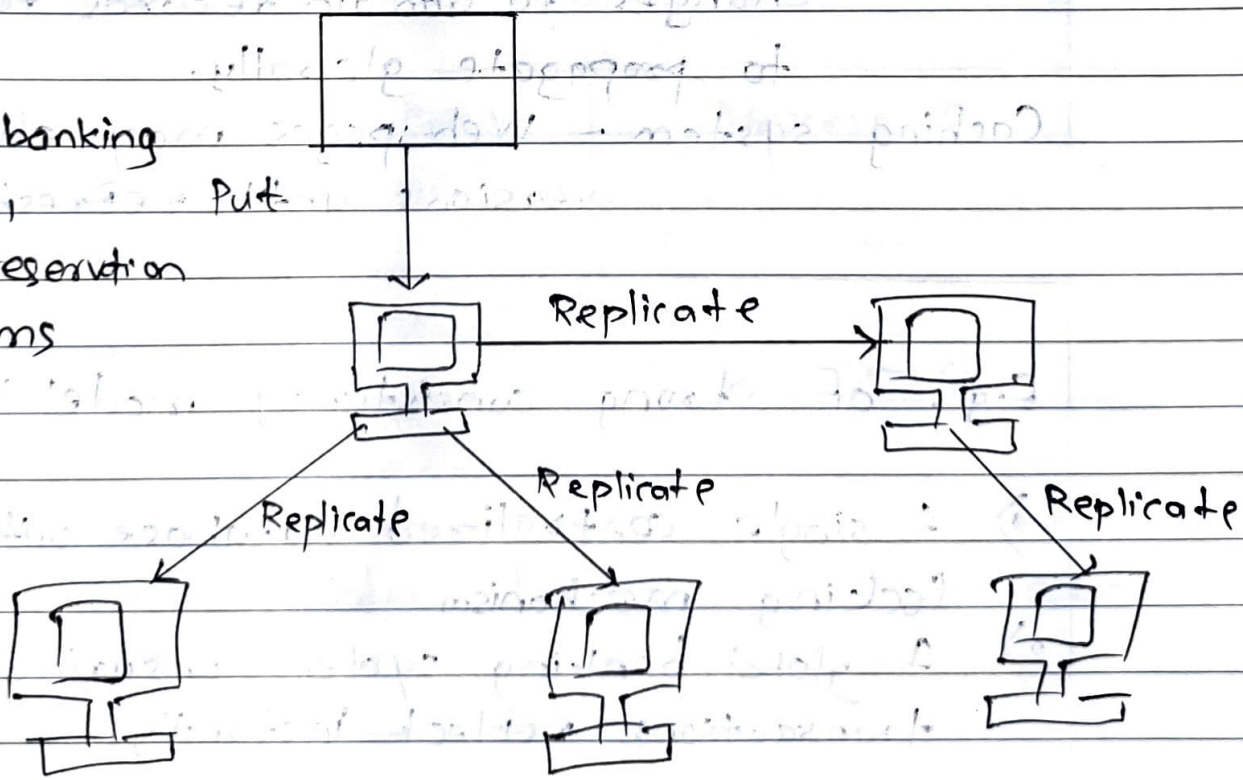
⇒

- In distributed systems, consistency models establish criteria for data synchronization and specify how users and applications should interpret data changes across several nodes.

1) Strong consistency model / strict

⇒ In the strongly consistent system, all nodes in the system agree on the order in which operations occurred.

- Reads will always return the most recent version of data; when an update occurs on one server, this model makes sure every other server in the system reflects this change immediately.



E.g., Online banking system, seat reservation systems

- This model provides the highest level of consistency but it can be slower and require more

resources in a distributed environment since all ~~resourc~~ servers must stay perfectly in sync.

2) Weak Consistency Model

⇒ A weakly consistent system provides no guarantee about the ordering of operations or the state of the data at any given time.

Relaxed approach

- Clients may see different versions of the data depending on which node they connect to.
- This model provides the highest availability and scalability but at the cost of consistency.
- For e.g.; social media websites,

DNS - changes to domain records take time to propagate globally.

Caching system - Web pages may show older versions until refreshed.

E.g. of Strong consistency model :-

1) A single, centralized database with strong locking mechanism
2) A global banking system ensuring all transactions reflect instantly.

Page No :
Date

Q.2 Explain sequential and release consistency models with examples

=)

1) Sequential Consistency Model :

=) It is a consistency model in distributed systems that ensures all operations across processes appear in a single, unified order.

- In this model, every read and write operation from any process appear to happen in sequence, regardless of where it occurs in the system.
- Importantly, all processes observe this same sequence of operations, maintaining a sense of consistency and order across the system.
- For e.g.,

Consider two processes P_1 and P_2, operating on a shared variable X :

Time	Process P_1	Process P_2
t_1	X = 10 (write)	
t_2		Read X → 10
t_3	X = 20 (write)	
t_4		Redd X → 20

- The read operations always see updates in a consistent order, (first X = 10 , then X = 20) but actual time between updates does not matter.

2) Release consistency

⇒ A release consistent system ensures that updates to shared data become visible only when a specific release operation occurs.

- A process must acquire a lock before modifying shared data.
- The changes are not immediately visible to other processes until a release operation is performed.
- For eg;

Consider two processes P_1 and P_2 using acquire(lock) and release(unlock) operations.

Time	P_1	P_2
t_1	Acquire lock	
t_2	X = 10 (write)	
t_3	Release Lock	
t_4		Acquire lock
t_5		Read X → 10
t_6		Release Lock

- P_2 cannot see P_1's update (X = 10) until P_1 release the lock.
- This ensures synchronization and proper ordering of shared memory accesses.

Page No.:
Date:

Q.3 Explain casual and FIFO consistency model with examples.

⇒

1) Causal Consistency Model

⇒ The causal consistency model is a type of consistency in distributed system that ensures that related events happen in a logical order.

- In simpler terms, if two operations are causally related, the system will make sure they are seen in that order by all users.
- However, if there's no clear relationship between two operations, the system doesn't enforce an order, meaning different users might see the operations in different sequences.
- For e.g;

P_1 :	$W(x)\ a$		
P_2 :		$W(x)\ b$	
P_3 :		$R(x)\ a$	$R(x)\ b$
P_4 :		$R(x)\ b$	$R(x)\ a$

- The read has been removed from P_2 (no casual relation between P_1 and P_2).
- Two writes are now concurrent and can read the x in any order without violating the rules.

Page No.:
Date:

2) FIFO Consistency Model

⇒ FIFO consistency is the next step in relaxing the consistency by dropping requirement of seeing the casually related writes in same order at all the processes.

- If a process writes multiple values, all other processes must observe them in the same order.
- However, writes from different processes may be seen in different orders by different observers.
- For e.g.,

P_1 : W(x)a			
P_2 :	R(x)a	W(x)b	W(x)c
P_3 :		R(x)b	R(x)a R(x)c
P_4 :		R(x)a	R(x)b R(x)c

- Here, W(x)b and W(x)c are the writes from same process P_2.
- Processes P_3 and P_4 sees these writes in the same order i.e. first b and then c.